

Tools — Sub-Agents

Spawn focused sub-agents that run synchronously or in the background. Prefer sub-agents over doing work inline: delegate exploration, builds, and planning.

Tool	Description
<code>new_agent</code>	Spawn a sub-agent (blocking or background).
<code>cancel_agent</code>	Cancel a running background sub-agent.
<code>list_agents</code>	List sub-agent tasks for the session.
<code>wait_agents</code>	Block until background tasks complete.
<code>agent_complete</code>	Terminal signal: the autonomous task is done.

System instruction: “Prefer sub-agents over doing work yourself. Use `background: true` when spawning more than one.”

Available agents: `explore` (fast, read-only codebase understanding), `build` (compile/test/fix), `plan` (implementation plans, read-only), `general` (full-capability fallback).

`new_agent`

Spawn a sub-agent to perform a focused task. **Both agent and task are required.** The agent is stateless — batch all related questions into one call with a comprehensive prompt.

Arguments

Argument	Type	Required	Description	Typical value
<code>agent</code>	string	yes	Agent name (<code>explore</code> , <code>build</code> , <code>plan</code> , <code>general</code> , or <code>custom</code>)	<code>"explore"</code>
<code>task</code>	string	yes	Specific prompt/instructions, including all needed context	<code>"Find all callers of X in src/"</code>
<code>background</code>	boolean	no	Run concurrently without blocking (default false). Use <code>true</code> whenever spawning more than one in the same response	<code>true</code>

Argument	Type	Required	Description	Typical value
<code>detached</code>	boolean	no	Only with <code>background: true</code> . Fire-and-forget: excluded from <code>list_agents</code> , not awaitable via <code>wait_agents</code> , completion NOT injected back into the session	<code>true</code>
<code>model</code>	string	no	Provider/model override	<code>"anthropic/claude-sonnet-4-20250514"</code>

Concurrency: at most 32 background tasks per session; use `wait_agents` to free slots before spawning more.

A **detached** task (`background: true`, `detached: true`) runs and shows in the Agents panel but is invisible to the delegation surface: `list_agents` omits it, `wait_agents` (with or without `task_ids`) never returns it, and its completion is reaped without a chat injection while still appearing in `tasks_snapshot`. Because nothing reads its reply body, a detached prompt whose deliverable is a report MUST name a file to write; the run's full output is additionally persisted to `log/subagents/<task-id>.md` on completion. The TUI `/spawn` command is the user-facing equivalent.

Example:

```
new_agent agent="explore" task="Summarise architecture in crates/ragent-agent/src/agent/" background=true
wait_agents
```

cancel_agent

Cancel a running background task spawned with `new_agent background=true`. Not for team tasks.

Arguments

Argument	Type	Required	Description
<code>task_id</code>	string	yes	Task identifier returned by <code>new_agent</code>

list_agents

List sub-agent tasks for the current session with status and result summary; completed tasks include an `output_file` path with the FULL untruncated report under `log/subagents/<task-id>.md`.

Arguments

Argument	Type	Required	Description	Typical value
status	enum	no	Filter by running / completed / failed / cancelled	—
task_id	string	no	Get details for one task	—

wait_agents

Block until one or more background tasks complete (preferred over polling `list_agents`). For team members use `team_wait` instead.

Arguments

Argument	Type	Required	Description	Typical value
task_ids	array	no	Specific tasks to await (default: all running)	["task-abc"]
timeout_secs	integer	no	Max seconds to wait before returning partial results	300

agent_complete

Terminal signal — the current autonomous task is fully done. Ends the session loop and returns control to the user. Only call when all requested outputs have been produced.

Arguments

Argument	Type	Required	Description
summary	string	yes	Concise summary of what was accomplished

Do NOT pass `task_id`, `team_name`, `result`, or `output`; to mark a *team* task done, use `team_task_complete` inside a team.